

Assessing the Relevance of Search Results Retrieved Through TF-IDF Compared to Meaning-Based Vector Embeddings

Overview

The aim of this project is to determine whether TF-IDF or vector search are better at finding relevant results for queries related to computing, technology and software. This sits in the domain of information retrieval combined with elements of natural language processing. I chose to investigate this as search engines are tools we use every day so it is important to understand which of the two major underlying processes behind them is optimal in terms of relevant results. For the data set, I created my own TF-IDF index and vector embeddings based on [GloVe embeddings](#). Although pre-trained indices were likely available, I determined it would be best to develop my own index and implement the code to better understand the algorithms and methods used in vector and TF-IDF search, as well as ensuring that the comparison was direct by using the same websites and testing methods.

Data Provenance

The data generating methodology consisted of the following: The indices and embeddings were both generated with my own crawler, using websites related to the topics as seeds. The crawler first visits all of the links on every site recursively and saves the content of each site. After this, it indices the sites by generating an average vector embedding and a TF-IDF value based on every word on the page. This data could be incomplete as some sites may have bot blockers so the crawler is unable to index them. The data is also limited by it's size as it only has 1.5 million sites indexed. For the data wrangling, all of the tokens need to be converted to lowercase to ensure they are all registered no matter their capitalisation. For the feature engineering, vector embeddings for each word of the query need to be averaged to give a single vector and when searching with TF-IDF, all of the tokens need to be taken into account. For each search request, cosine similarity is used to find the best result for vector search and for TF-IDF, the results are ranked by the highest average TF-IDF values for all of the query tokens. After creating the index, no further operations are required and the data is ready for analysis.

Evaluation Methodology

For the analysis, I created a set of queries including ones that are challenging for both TF-IDF and vector search to test performance in different areas. For example, "speedy programming languages" for TF-IDF or "plasma desktop" for vector. Using these queries, I added a page dedicated to quick evaluation of each method's performance. It goes through the set of queries and shows a list of the top three results from each search method which are randomised to avoid bias. The judge then ticks the results which are relevant and goes to the next query. These results are then presented after evaluation on a summary page which includes a per-category breakdown of where each method struggled and succeeded, a table of the results, as well as the overall success of the methods.

Results

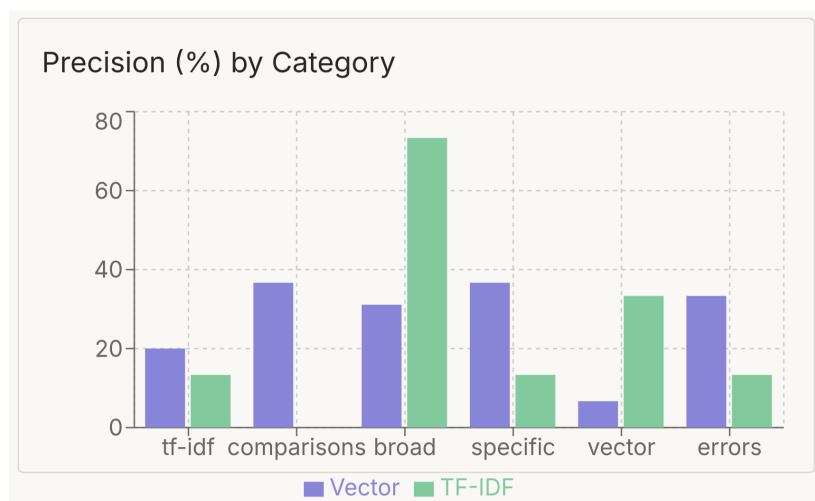


Figure 1: Performance by Category

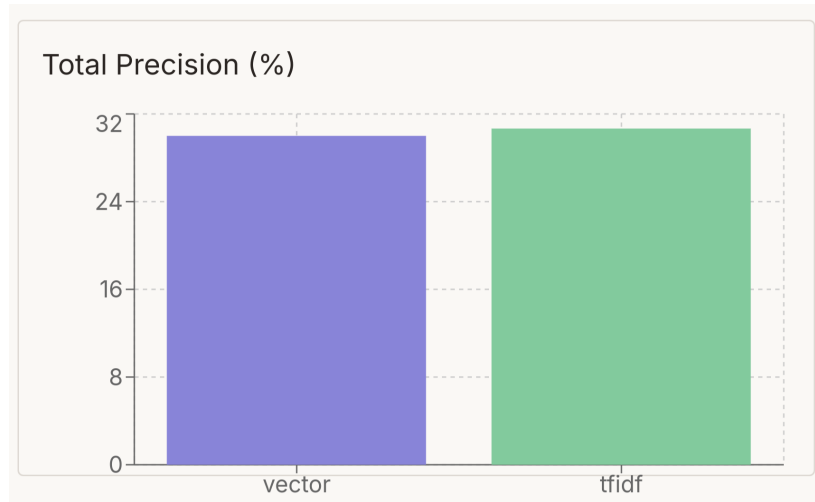


Figure 2: Overall Performance

In [Figure 1](#) we can see that TF-IDF performs exceedingly well with "broad" queries with a precision of over 73%. This is significantly better than vector which got a 31% ranking on this category. TF-IDF also beats vector search in the category that is for tricky for vector with a precision of approximately 33% compared to 7%. Although vector falls short in these two categories, it excels in all of the other categories where TF-IDF doesn't perform as well like comparisons, more specific queries and debugging errors. On average it performed 22% better than TF-IDF in these categories. For the "errors" category, vector's success is surprising as I originally predicted that there would be more token misses or poor understanding of technical language (like "hooks") by the pre-trained embeddings. Overall, as can be seen in [Figure 2](#), TF-IDF is narrowly more performant but, being within a margin of error, they are practically the same overall. Vector performs more consistently in most categories, other than the edge cases designed to target the limits of vector search, whereas TF-IDF is less consistent, performing surprisingly well in "broad" queries but completely failing in the "comparisons" category. Ultimately, the method best suited for relevant results is based on what the user is searching for. For research, general searches and rare keywords, TF-IDF would be the best choice but for specific, targeted queries, comparisons and finding results through description without knowing specific vocabulary, vector search is more suitable.

Strengths and Weaknesses

As I used the same sites for both TF-IDF and vector search, one of the central strengths of the project is it provides a good direct comparison between both search algorithms as the same set of websites are used. In my current implementation of TF-IDF, stopwords are included which could skew the results as similar words have not been converted to the root word. To improve the TF-IDF performance, it would have been good to remove these stopwords and replace similar words (like running → run) for the index. For vector search, there may be many token misses and incorrect understanding of regular words used in a technical context. To improve this, creating custom trained vector embeddings would be beneficial as the vectors would be more suited to the topics of interest and they would include rare words. Perhaps the [GloVe project's generator](#) (which created the pre-trained embeddings that I used) could be used to generate these. In a real-world search engine, there would be other factors that affect the ranking of pages like popularity but this is not in the scope of this project. To improve the size of the index, it would have been better to use a desktop computer for a faster computation of the indices as it took over a week of continuous operation to create the index.

Ethical, Practical and Societal Implications

Having run the crawler and indexer for over a week and only yielding 1.5 million sites made demonstrated the vast amount of computational power and energy required to create a search index. Google has about 400 billion sites in its index. At the speed of the laptop that was used for this project, this would take over 10 000 years of indexing to create and that's not even considering the size of the database. For 1.5 million sites, the size of storing the documents, the index, and everything else combined, adds up to 310GB. At this rate, Google's index would require 72 petabytes. With modern day prices and considering double for redundancy, this would be worth £30 million in SSDs but this is definitely an underestimate as Google also stores descriptions of the sites and other information. Some might say that this money could be put to better

use but the quick and easy access it provides to facts, information and answers could be seen as more helpful for science, research and education. The environmental impact of the energy used to create the index, run the queries and to build the hardware required is substantial and should also be considered.

Key Takeaways

The most impactful lesson this project taught me was to not reject AI and to harness the power that it can provide. Before the project, I had a negative outlook on AI but through practical experience with [OpenCode](#) (an AI agent), I now understand the productivity and efficiency benefits it can provide when programming. Without AI, the project would likely have taken too long and I would not have any meaningful results. If I had used AI sooner, this would have created the leeway to develop a larger index and more expansive results which would have likely improved the precision of the models. In the future, I will continue this project, improve the user interface and include more features like ranking by popularity, Wikipedia cards and AI overviews.

Future of Search Engines

As artificial intelligence becomes more popular, traditional search engines are becoming obsolete. Google is integrating AI more and more into their search results by providing an AI summary and an AI mode. Websites are no longer practicing search engine optimisation (SEO) but instead they focus on generative engine optimisation (GEO) which is the practice of optimising websites to improve visibility in answers provided by generative AI. In my opinion, this shift in the way we find information has many implications on the access of fact-checked truth. Traditional search engines force you to navigate to the website and read it to discover the information which forces you to scrutinise the site and ensure that it is correct. With the rise of AI finding the data for us, people are less inclined to manually check if the information is true, instead opting to just trust what the summary tells them. Companies creating generative engines need to consider the sources that the AI can use as certain social media sites have given potentially dangerous information which the AI summarises as if it's fact. This highlights the importance of still having the skills to verify information obtained online and how search engines are still relevant to discover new websites, research obscure topics and to critically evaluate controversial topics which may be biased by AI.